PROJECT PORTFOLIO

프로젝트 포트폴리오

프로젝트 목록

- 제주패스 렌트카 열쇠 보관함
- 삼성전자 DS 공사작업 무인보관함
- 정부과제 성동구 민원서식 도우미

PROJECT 01

제주패스 렌트카 열쇠 보관함 프로젝트

프로젝트 소개

프로젝트	**KeepUs Go** — ThingsBoard 기반 무인보관함 멀티테넌트 관제 앱. 제주패스는 B2B 결제형 렌트카 열쇠보관 고객사 사례
기간	2025.04 – 2026.06
역할	Flutter 크로스플랫폼 앱 설계·구현 (View / Controller / Repository), ThingsBoard·커스텀 백엔드·Firebase 연동, 운영 UX
스택	Flutter / Dart, GetX, thingsboard_client, Dio, Firebase Auth·Storage, Syncfusion Charts, Trina Grid, flutter_map
플랫폼	Web · Android · iOS 단일 코드베이스
핵심	ThingsBoard Admin을 고객에 노출하지 않고, 관제 앱은 REST·Telemetry WebSocket·Server-side RPC로 실시간 모니터링·원격 제어. 키오스크(엣지)만 TB와 MQTT. 테넌트 플래그(`enable_payment` 등)로 결제형·자산 대여형 제품을 코드포크 없이 분기

담당 업무와 성과

담당 업무

- Flutter + GetX로 Web·Android·iOS 관제 앱을 단일 코드베이스로 설계·구현
- ThingsBoard REST·Telemetry WebSocket·Server-side RPC로 상태 실시간 반영 및 원격 제어(문 열기·충전·LED 등)
- 집계·메일·AI SSE는 Custom Backend API로 분리하고, ThingsBoard JWT 공유·401 silent refresh로 인증 일원화
- 테넌트 기능 플래그(`enable_payment`, `handle_inventory`)로 메뉴·도메인을 분기해 코드포크 없이 고객사별 제품 변형 대응
- 다국어(ko/en/ja/zh), 대시보드·지도·알람·보관함 셀·결제 이력 UI 전반 담당

주요 성과

- 단일 앱으로 웹 관제와 현장 모바일(QR·푸시) 시나리오를 함께 제공
- 모델별 이력 사진 누락(텔레메트리 ts 분리)을 정규화 유틸·단위 테스트로 해소
- Customer 계정 QR 등 silent 권한 실패를 API 스코프 가드로 제거
- 웹에서 Firebase Storage 본문 fetch가 CORS로 막힐 때 iframe 등 폴백으로 인프라 대기 없이 조회 가능하도록 개선
- WebSocket + JWT refresh 충돌을 앱 레벨 구독 레지스트리로 완화
- 키오스크 수정이 어려운 구간에서 앱 쪽 방어적 처리·폴백으로 운영 이슈를 흡수

시스템 구조

```
KeepUs Go (Flutter)
  View → Controller → Repository → API Clients
       | REST/WS/RPC      | REST(+SSE)     | Auth/Storage
       ▼                  ▼                ▼
  ThingsBoard (IoT SSOT)  Custom Backend  Firebase → GCS (사진·로그)
       ▲ MQTT only
  Kiosk Device
```

시스템	역할
ThingsBoard	Device · Attribute · Telemetry · Alarm · Relation · RPC의 SSOT
Custom Backend (KUB)	집계 · 메일 · AI 챗봇 등 ThingsBoard에 없는 제품 API (TB JWT 공유)
Firebase / GCS	키오스크가 올린 이용자 사진 · 일별 로그 조회

도메인: Tenant → Customer → Asset(지점) → Device(키오스크). 함(Cell)은 별도 Device가 아니라 Device flat attribute (`cell_*`).

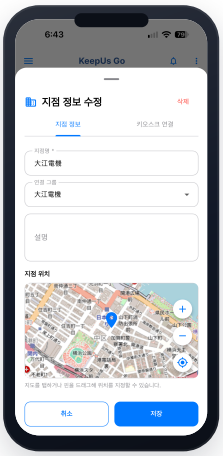
설계 판단

판단	이유
Flutter 단일 코드베이스	웹 관제와 현장 모바일을 한 팀·한 도메인으로 유지
GetX	실시간 필드 갱신(Obx)·라우팅·DI 생산성. 루트 전면 Obx는 피하고 화면별 구독
Cell flat model	N함=N Device면 엔티티·구독·비용이 폭증 → 1 Device = N cells
ThingsBoard + Custom Backend (KUB) 분리	IoT 허브에 집계·AI를 억지로 넣지 않음. JWT는 ThingsBoard와 공유
방어적 처리	SDK null, telemetry ts 어긋남, CORS, Customer API 스코프를 Repository·Util·폴백에서 처리

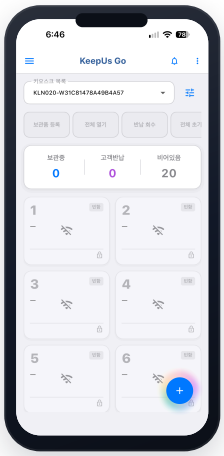
주요 화면



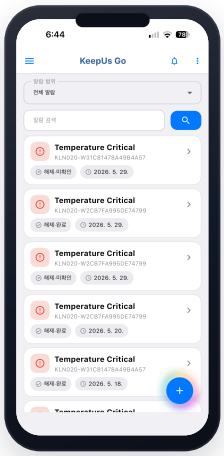
대시보드



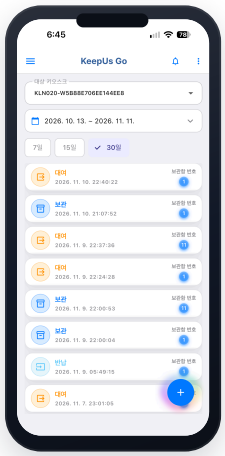
지점 관리



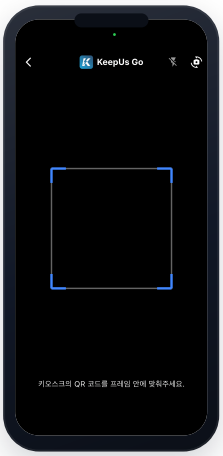
보관함



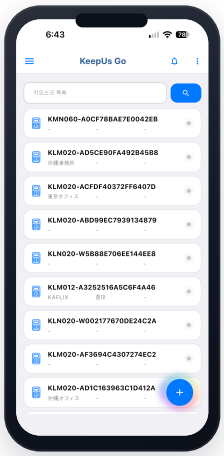
알람



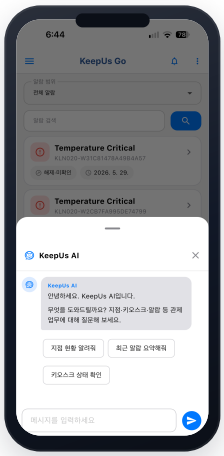
사용 이력



QR 스캔



키오스크 목록



AI 챗봇 기능

문제 해결과 개선

이력 사진 미표시 — 텔레메트리 ts 분리

문제
이력에 이용자 사진이 안 뜸. KLN은 되고 KLM만 안 되는 편차로 Storage 장애로는 설명되지 않음.

대응
행의 picPath 공백과 Storage 로드 실패를 분리. action과 pic_path가 다른 ts 버킷으로 송신됨을 확인 후 ≤5초·cell id 조건으로 병합 유틸·단위 테스트 적용.

결과
모델별 “사진은 올렸는데 앱에만 안 보인다” 이슈 해소. CS·키오스크 팀과 ts 버킷 기준으로 원인을 공유할 수 있게 됨.

관제 UI 버그처럼 보이던 문제의 원인은 엣지 송신 계약의 느슨함이었습니다. 키 존재 여부, 스토리지 파일 존재 여부, 시계열 묶음 방식을 분리 검증한 뒤, 병합 윈도우를 운영 송신 간격에 맞춰 5초로 수렴시켰습니다.

배움. 증상만 보고 Firebase/앱/키오스크를 단정하지 말고 레이어를 갈라 검증할 것. 병합 윈도우는 길면 오매칭·짧으면 누락이라는 트레이드오프가 있다.

Customer QR silent failure — API 스코프 누락

문제
Tenant Admin은 QR이 되고 Customer는 "기기를 찾을 수 없습니다". 크래시 없이 실패해 권한 문제로 보이지 않음.

대응
`getAllDeviceInfos()`에 customerId 미전달로 Tenant API를 타던 경로를 확인하고, Customer 세션이면 customerId를 넘기도록 수정. 동일 패턴 로드 경로에 가드 보강.

결과
할당된 함만 보고 QR로 바로 이동하는 현장 시나리오 복구. 탭 숨김과 API 스코프 정합을 분리해 보는 기준으로 정리.

메뉴에서 Customer에게 Branch를 숨긴 것만으로 권한이 맞다고 보기 쉬웠습니다. 권한 회귀는 403이 아니라 빈 결과로 위장할 수 있어, 화면 노출 여부와 호출 API의 허용 범위를 함께 확인하도록 구조를 맞췄습니다.

배움. Admin 해피 패스만 검증하면 권한 회귀를 놓친다. "탭이 보이는가"와 "이 버튼이 치는 API가 Customer에게 허용되는가"는 다른 질문이다.

실시간 관제 안정화 — 좌표 소스 · CORS · WebSocket

지도 좌표
REST 초기값은 CLIENT_ATTRIBUTE, WS Latest Telemetry가 동명 키를 덮어써 위치가 흔들렸습니다. 장기 SSOT는 CLIENT이지만 당시 키오스크 CLIENT 갱신이 어려워, 초기 CLIENT fallback + WS Telemetry 덮어쓰기로 앱 정책을 정리하고 구독 누락을 수정했습니다.

웹 Storage CORS
웹에서 Firebase Storage 파일 본문(XHR) fetch가 GCS CORS로 실패하는 경우가 있었습니다. list/download URL 경로와 본문 fetch 경로를 구분해, 실패 시 iframe 등 폴백으로 인프라 대기 없이 운영을 이어갔습니다.

WS + JWT refresh
토큰 갱신 후 WS close 충돌이 났습니다. Client WS reset 전 활성 디바이스 구독을 앱 레지스트리에서 먼저 dispose하도록 정리했습니다.

배운 점과 개선점

잘한 점
- ThingsBoard 콘솔을 고객에 노출하지 않고, 엣지 MQTT와 관제 REST/WS/RPC 경로를 분리
- Cell flat model과 ThingsBoard·Custom Backend 경계로 IoT 허브 과부하를 피함
- 송신 패턴·권한 스코프·CORS처럼 운영에서 드러난 문제를 앱 쪽 방어적 처리·폴백으로 정리

개선하고 싶은 점
- ThingsBoard SDK를 Repository 인터페이스 뒤로 더 일찍 두고, Tenant/Customer API 분기 규칙을 공통 체크리스트로 고정했어야 함
- GetX는 현 규모에 유효하나 테스트 비중이 커지면 상태관리 재평가가 필요
- GCS CORS 인프라 정비와 Crashlytics 등 관측성 표준화는 남겨 둔 과제

PROJECT 02

삼성전자 DS 공사작업 무인보관함 프로젝트

프로젝트 소개

프로젝트	**삼성전자 DS 공사작업 무인보관함 관제** — 공사현장 모바일타워(무인보관함)의 JSP 레거시 관제를 Flutter 모듈형 웹으로 전환한 B2B 관제 콘솔
기간	2024.02 – 2025.05
역할	Flutter 관제 웹 설계·구현, GetX 기반 MVVM(화면·상태·통신) 분리, JSP 기능 이전, 실시간 조회·이벤트 전달 구조 개선
스택	Flutter / Dart, GetX (MVVM), Dio, PlutoGrid, Syncfusion Charts, flutter_dotenv, Excel, WebSocket(이벤트 계층)
플랫폼	Web (관제 콘솔) — 온프레미스/Tomcat·데모 서버 `base-href` 배포
핵심	레거시 화면·로직을 View / Controller·ViewModel / Repository로 분리한 관제 콘솔. 도메인: 타워(키오스크)·셀·대여/반납/보관·사용 이력·이슈·계정·역할·회사 스코프. 관제 웹 조회는 REST(`/v1`)·JWT, 상태 변경 즉시성은 이벤트 전달 계층에서 담당

담당 업무와 성과

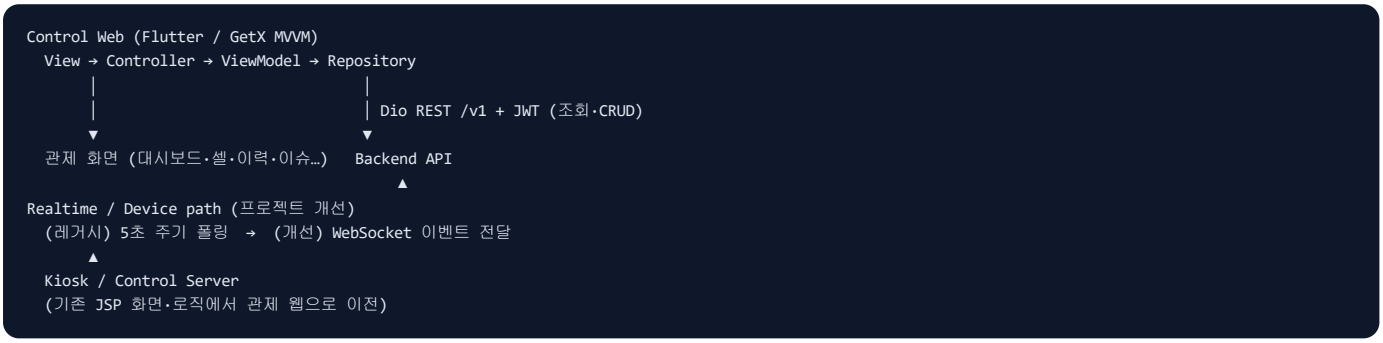
담당 업무

- Flutter Web 관제 앱 설계·구현 — 대시보드·보관함·사용 이력·알림·계정·타워 마스터
- GetX MVVM으로 View → Controller → ViewModel → Repository 책임 분리 및 Dio REST(/v1)·JWT 연동
- 기존 JSP 관제 화면·로직 분석 후 동일 업무 흐름을 모듈형 Flutter로 이전
- 역할(superAdmin/admin/manager/operator)·회사 스코프에 맞춘 메뉴·데이터 필터
- PlutoGrid·차트·엑셀 import/export, 세션 만료·API 예외 UX 일원화
- 5초 주기 폴링을 WebSocket 이벤트 전달로 전환해 상태 변경 시점 갱신 구조 개선

주요 성과

- JSP 중심 레거시 화면·로직을 Flutter 모듈형으로 재구성해 화면·상태·데이터 통신 계층의 책임을 분리 — 기능 추가 시 어디를 손댈지 경계가 분명해짐
- 공통 화면 셸·재사용 컴포넌트로 신규 관제 기능을 기존 구조에 반복 추가할 수 있는 개발 기반 구축
- 기존 5초 주기 폴링을 WebSocket 기반 이벤트 전달로 전환해, 상태 변경이 발생한 시점에 데이터를 갱신
- 반복 요청 중심 조회를 실시간 이벤트 스트리밍 구조로 바꿔 불필요 API 호출을 줄이고 관제 화면의 정보 즉시성 향상

시스템 구조



계층	역할
View	대시보드·보관함·이력·알림·계정·타워 등 관제 UI (NavigationRail·그리드·차트)
Controller · ViewModel	화면 상태·권한 분기·로딩. ViewModel이 Repository를 감싸 비즈니스 호출을 모음
Repository · REST	Dio로 /v1 인증·키오스크·셀·이력·이슈·계정 API. Bearer JWT, 401 시 세션 만료 UX
이벤트 계층	상태 변경 시점의 WebSocket 이벤트 전달 — 관제 웹의 REST 조회와 경로를 분리
권한 · 테넌트	역할·회사별 관리 가능 타워 집합을 세션에서 공유해 화면·데이터 스코프 정합
레거시	JSP 관제 기능을 Flutter 모듈로 이관. UI 복제가 아니라 계층 경계를 다시 잡음

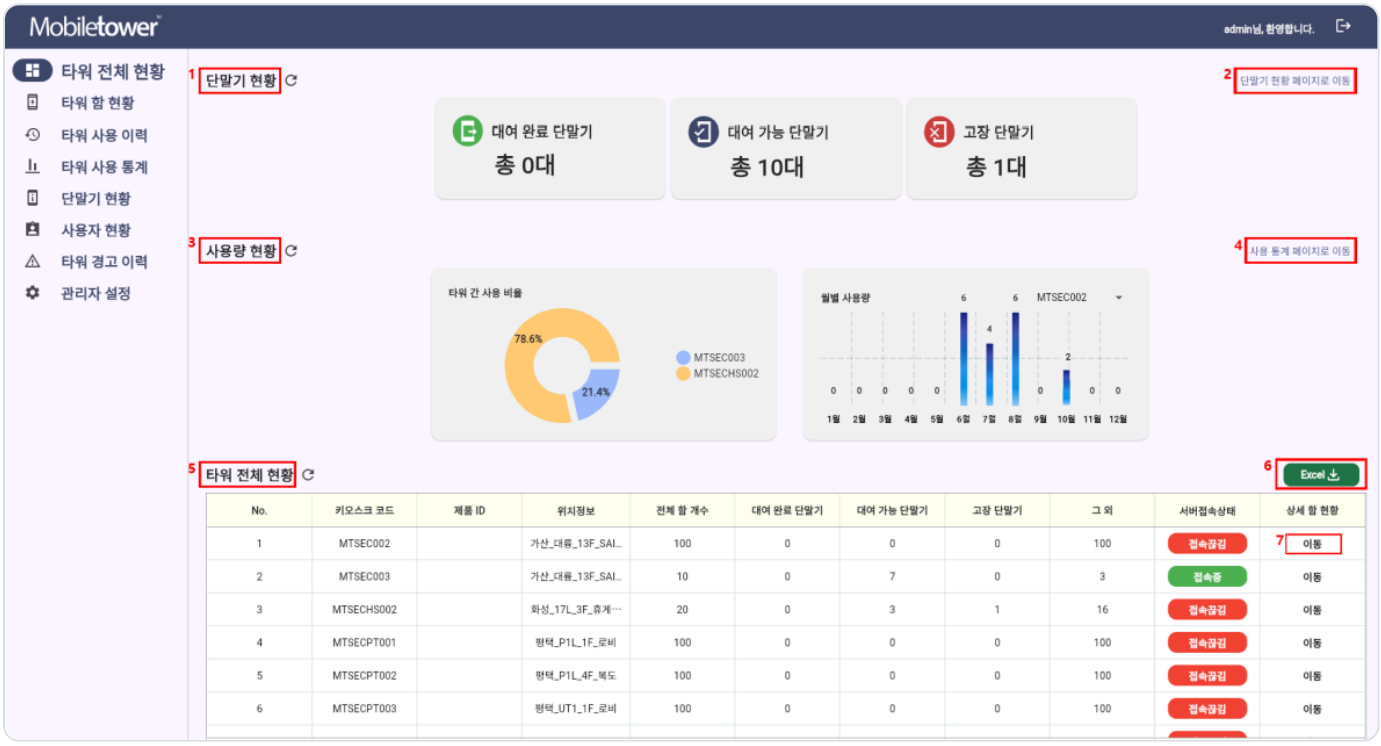
설계 판단

판단	이유
Flutter + GetX MVVM	웹 관제를 단일 코드베이스로 유지하면서 View / 상태 / 통신을 분리해 JSP 대비 수정·확장 비용을 낮춤
공통 셸 · 재사용 컴포넌트	신규 관제 화면을 같은 레이아웃·그리드·알림 패턴으로 붙여, 화면마다 구조를 다시 짜지 않음
폴링 → WebSocket 이벤트	고정 주기 조회는 변화 없어도 호출이 쌓임. 변경 시점 이벤트로 바꿔 부하를 줄이고 즉시성을 맞춤
REST 조회와 이벤트 경로 분리	마스터·이력·CRUD는 REST로 명확히 두고, 즉시성이 필요한 상태만 이벤트 계층에 맡김

주요 화면



Before · JSP 레거시 관제



After · Flutter 개선 관제

문제 해결과 개선

JSP 레거시 → Flutter 모듈형 이관

문제
화면·비즈니스·통신이 JSP에 섞여 있어, 기능 추가·수정 때마다 영향 범위가 넓고 동일 패턴을 화면마다 다시 구현하기 쉬웠음.

대응
View / Controller·ViewModel / Repository로 책임을 나누고, 공통 화면 셸·그리드·알림 등 재사용 컴포넌트를 먼저 고정한 뒤 JSP 업무 흐름을 이전.

결과
신규 관제 기능을 같은 모듈 패턴으로 반복 추가할 수 있는 기반을 만듦. 면접에서는 「이관 = UI 이식」이 아니라 「책임 경계 재설계」로 설명.

레거시를 픽셀 단위로 옮기는 대신, 관제에 필요한 도메인(타워·셀·이력·이슈·계정)을 API·상태·화면 계층에 나눠 올렸습니다. 이후 대시보드·보관함·이력 등은 공통 Navigation·그리드 관례 위에 쌓는 방식으로 확장했습니다.

배움. 레거시 이관의 성패는 화면 완성도보다, 다음 기능을 같은 비용으로 붙일 수 있는 경계가 생겼는지에 있다.

5초 폴링 → 이벤트 기반 갱신

문제
고정 5초 주기로 상태를 조회하면 변화가 없어도 요청이 반복되고, 변경 직후에도 다음 주기까지 화면 반영이 늦을 수 있음.

대응
상태 변경 시점의 WebSocket 이벤트 전달로 전환. 마스터·이력·CRUD 등 조회·조작은 REST로 두고, 즉시성이 필요한 경로만 이벤트 계층으로 분리.

결과
불필요 API 호출을 줄이면서 관제 정보의 즉시성을 높임. 「자주 새로고침」이 아니라 「필요할 때 온다」는 설계로 정리.

폴링은 구현이 단순하지만 부하와 지연이 구조적으로 남습니다. 이벤트 전달로 바꾼 뒤에는 관제 화면의 즉시성과 서버 호출 패턴을 함께 말할 수 있게 되었습니다. 관제 웹의 목록·CRUD 조회는 REST(`/v1`)로 유지해, 실시간 계층과 조회 계층의 역할을 면접에서도 구분해 답할 수 있습니다.

배움. 「화면이 자주 갱신된다」와 「데이터가 변경될 때 온다」는 다른 요구다. 조회 REST와 이벤트 경로를 나누면 둘을 동시에 만족시키기 쉽다.

배운 점과 개선점

잘한 점
- JSP를 Flutter 모듈형으로 옮기며 View·상태·통신 경계를 먼저 고정해 확장 가능한 관제 기반을 만듦
- 폴링과 이벤트·REST 조회 경로를 분리해 즉시성과 명확한 CRUD를 함께 가져감
- 역할·회사 스코프와 세션(401) UX를 관제 화면 전반에 맞춰, 멀티테넌트 노출 실수를 줄임

개선하고 싶은 점
- 원격 열기·리부트 등 일부 제어 UI는 확인 흐름만 있고 API 연동이 남아 있어, 관제→현장 제어 완결성을 더 닫고 싶음
- 셀 통계를 타워별로 순회 호출하는 집계(N+1)와 토큰 silent refresh·관측성(크래시·성능)은 이후 과제로 남김

PROJECT 03

성동구청 민원서식 도우미 프로젝트

프로젝트 소개

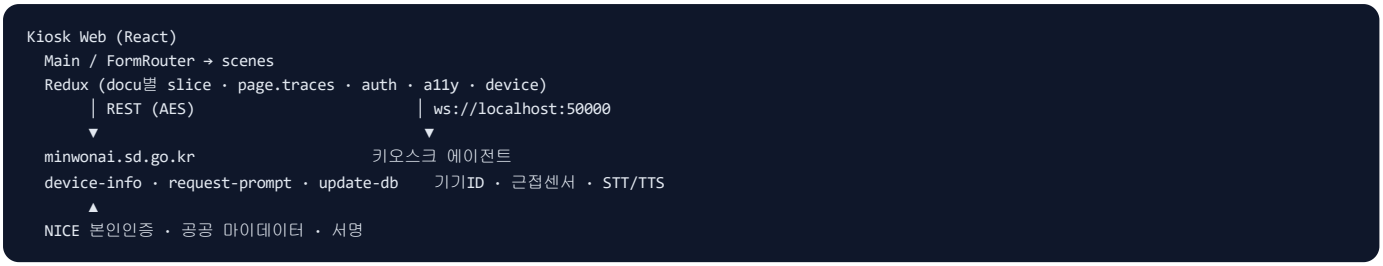
프로젝트	**성동구청 민원서식 도우미** — 정부과제 키오스크. 창구 민원 서식을 단계형 화면으로 작성하고 음성·접근성으로 제출까지 이어 주는 무인 창구 (`minwonai.sd.go.kr`)
기간	정부과제
역할	React 키오스크 프론트 설계·구현 (서식 위저드, 음성·접근성, NICE·마이데이터·제출 연동). Figma·Photoshop으로 와이어프레임·아이콘·주요 그래픽 협업
스택	React 18, Redux Toolkit, React Router 6, styled-components, Axios, WebSocket, AES256, NICE 본인인증, 공공 마이데이터
플랫폼	키오스크 웹 (1080px 세로) · 태블릿
핵심	전입·출생·사망 등 서식 15종을 공통 셸 위의 단계 화면으로 제공. 서식별 Redux, 키오스크 로컬 WebSocket(음성·센서·기기), HTTPS 제출(AES). 스크린리더·TTS·돋보기·큰 글씨·낮은 화면으로 무인 이용을 보조

담당 업무와 성과

담당 업무
- 서식 15종 라우팅·단계 화면 — 전입세대 열람, 전입신고, 주민등록증 재발급, 문화이용권, 우대 교통카드, 복지 요금감면, 여성청소년 생리용품, 복지급여 계좌변경, 외국인 체류지 변경, 장애인등록증 재교부, 출생신고, 주민등록 정정, 사망신고, 장애인 주차표지, 임산부 가사돌봄
- 서식별 Redux 슬라이스(`docu1`–`docu15`)와 페이지 방문 이력(`traces`)으로 분기 단계의 입력·뒤로가기 관리
- 키오스크 로컬 WebSocket으로 기기 인증, 근접센서, STT 결과→서식 이동, TTS 안내 연동
- 스크린리더, 돋보기, 낮은 화면, 큰 글씨, 다국어(KOR/ENG/CHN/JPN), 5분 무입력 초기화
- NICE·모바일 QR 본인인증, 공공 마이데이터 동의, 서명 후 AES256으로 `update-db` 제출
- 디자인 리소스가 부족한 현장에서 Figma·Photoshop으로 와이어프레임, 아이콘, 주요 그래픽 시안을 협업 제작하고, 구현 가능성과 데이터 구조를 함께 검토

주요 성과
- 민원 현황, 처리 상태, 통계 데이터를 시각화하여 데이터를 개별 확인하지 않고 하나의 화면에서 업무 현황을 파악할 수 있도록 구축
- Redux 상태 구조와 렌더링 범위를 재정비하여 데이터 변경 시 불필요하게 전체 화면이 갱신되는 구조 개선
- Node.js·Express 기반 WebSocket 서버를 도입하여 주기적 API 조회 대신 변경 이벤트 중심으로 알림과 데이터를 전달하도록 개선
- 디자인 리소스가 부족한 현장 상황에서 Figma와 Photoshop 스킬을 활용해 와이어프레임, 아이콘, 주요 그래픽 시안 협업 제작
- 개발 단계에서 구현 가능성과 데이터 구조를 함께 검토하여 불필요한 재작업을 줄이고 디자인과 개발 사이의 의사결정 속도 향상

시스템 구조



계층	역할
화면 셸	/ 메인 메뉴, /:index는 FormRouter가 서식을 선택. 단계는 page.currentPage로 FormXX/scenes/PageN을 로드
Redux	서식 본문은 docuN, 공통은 페이지 이력·인증·언어·모달·TTS/STT·스크린리더·센서·기기·AES 키
로컬 WebSocket	ws://localhost:50000 키오스크 에이전트. 기기 ID, 근접센서, STT 이벤트, TTS 연동. 음성 레벨(RMS)은 300ms throttle
백엔드 REST	/api/device-info, /api/request-prompt, update-db. 이름·서식 JSON은 AES256
본인확인	NICE, 모바일 QR. 공공 마이데이터 동의·조회, 서명 캔버스, 가상 키패드, 주소·한자 성명 검색

설계 판단

판단	이유
서식마다 docuN 슬라이스	전입·출생·사망처럼 필드 트리가 달라 서식 단위로 상태를 분리. 페이지·인증·언어·모달만 공통
page.traces	뒤로가기는 실제 방문 스택 기준. 동의·위임·마이데이터 같은 분기 단계는 방문 이력으로 복귀
WebSocket과 REST 분리	음성·센서·기기 이벤트는 로컬 에이전트, 프롬프트 합성과 DB 제출은 HTTPS. RMS는 throttle로 구독 빈도를 제한
Support · ScreenReader · ForeignLang	메뉴 노출, 스크린리더, 외국어(체류지 변경)를 서식 메타 플래그로 둠
제출 시 이름·본문만 AES256	기기 ID·서식 ID·단계·연령·성별은 운영 키로 평문. 개인정보 필드와 운영 키를 나눔
props · redux · localStorage	단계 안 임시값은 props, 화면이 반응할 값은 redux, 새로고침 뒤에도 남을 기기 정보는 localStorage

문제 해결과 개선

서식별 상태와 단계 이력

문제
전입·출생·사망은 필드 구조가 다르고, 동의·위임·마이데이터처럼 단계가 갈라져 단순 페이지 번호만으로는 입력을 유지하기 어려움.

대응
서식마다 docuN 슬라이스를 두고, 페이지 이동은 traces에 쌓아 뒤로가기가 직전 방문 단계로 돌아오게 함. 공통 UI 상태는 별도 슬라이스로 분리.

결과
한 서식의 필드 수정이 다른 서식 상태를 건드리지 않음. 분기 화면에서도 이전 단계의 입력 맥락을 유지한 채 되돌릴 수 있음.

서식 본문과 화면 크롬(언어, 모달, 음성, 기기)을 스토어에서 갈라 두면, 어떤 값이 단계 사이에 살아 있어야 하는지를 면접에서 슬라이스 단위로 설명할 수 있습니다. 뒤로가기는 번호 감소가 아니라 방문 스택 pop입니다.

배움. 필드 트리가 다른 화면을 한 객체에 모으면 초기화와 구독 범위가 함께 넓어진다. 서식 단위 슬라이스와 방문 이력을 나누면 입력 수명과 이동 규칙을 따로 말할 수 있다.

음성으로 서식 진입

문제

무인 키오스크에서 서식 이름을 읽어 해당 화면으로 들어가야 하고, 인식 실패·타임아웃 때는 이용자가 멈춘 것처럼 보이면 안 됨. 사람이 다가온 시점에도 대기 화면이 깨어나야 함.

대응

STT `UPDATE_RESULT.url`(서식 테이블 코드)을 `document_forms` 인덱스로 바꾼 뒤, 안내 음성이 끝나면 `/${index}`로 이동. 실패·타임아웃·거절은 TTS로 되돌림. 근접센서 `detected`는 대기 영상을 재생.

결과

음성 인식 결과가 메뉴 클릭과 같은 진입 경로가 됨. 인식이 비어도 음성 안내로 루프가 닫히고, 이용자 접근은 센서 이벤트로 대기 화면과 연결됨.

화면 이동은 인식 직후가 아니라 안내 음성 `onended` 이후입니다. 음성이 끊긴 채 화면이 바뀌지 않도록, STT 이벤트와 TTS 재생 순서를 WebSocket 메시지 처리 안에 맞춰 두었습니다. 음성 레벨은 300ms throttle로 받아 레벨 표시가 매 프레임 구독으로 퍼지지 않게 했습니다.

배움. 음성 UX는 인식 성공률만이 아니라, 성공 시 이동 시점과 실패 시 안내가 한 사이클인지가 체감 품질을 가른다.

무인 창구의 접근성과 세션

문제

키오스크는 이전 이용자의 돋보기·큰 글씨·언어·스크린리더가 다음 사람에게 남기 쉽고, 음성 안내와 스크린리더가 동시에 울릴 수 있음. 등록되지 않은 기기에서의 작성도 막아야 함.

대응

제출 전 스크린리더가 켜져 있으면 끄고 STT와 겹치지 않게 함. 돋보기·낮은 화면·큰 글씨·비한국어는 5분 무입력이면 메인 기본값(한국어)으로 복귀. `/api/device-info`에서 기기 ID가 없으면 토스트로 이용을 중단.

결과

접근성 옵션은 한 세션 안에서만 유지되고, 다음 이용자는 기본 화면에서 시작. 미등록 기기는 서식 작성 전에 걸러짐.

스크린리더 지원 여부는 서식 메타 `ScreenReader`로, 외국어 서식은 `ForeignLang`(체류지 변경)으로 나눕니다. 돋보기는 화면을 html2canvas로 잡아 확대하고, 포커스는 `react-aria` FocusScope로 가둡니다. 개발 중에는 axe-core로 접근성 진단을 돌렸습니다.

배움. 무인 단말의 접근성 기능은 켜는 경로와 끄는 경로가 짝이어야 한다. 타임아웃 초기화와 제출 시 리더 정지가 그 짝이다.

배운 점과 개선점

잘한 점

- 서식 메타와 docuN 슬라이스 경계로 15종을 한 셸(헤더·푸터·모달·라우터) 위에 올림
- 음성 이벤트를 안내 음성 종료 뒤 화면 이동과 맞춰, 인식 성공·실패·타임아웃이 각각 다음 행동을 갖게 함
- 개인정보(이름·서식 JSON)와 운영 키(기기·서식·단계)를 제출 payload에서 구분해 AES 적용 범위를 한정

개선하고 싶은 점

- 일부 서식 페이지가 렌더 중에 `dispatch(setFormId)`를 호출함. 단계 진입 effect에서 한 번만 설정하는 편이 맞음
- 메인 메뉴가 `store.subscribe`로 음성·기기 상태를 읽음. 필요한 필드만 `useSelector`로 구독해 렌더 범위를 더 좁히고 싶음
- 단계 화면을 동적 `require`로 바꿈. 서식별 라우트 맵으로 두면 없는 단계와 로딩 실패를 미리 드러낼 수 있음